

RLLBC BPA3: Training on the RWTH HPC Cluster

This guide explains how to run training jobs for **Bonus Point Assignment III (BPA3)** on the RWTH HPC cluster.

The guide is optional. It is intended for students who want to run longer training jobs or compare several random seeds without blocking their local machine.

Running jobs on the cluster uses core hours. Every student has a personal monthly quota that they can use.

Intended workflow

Initial workflow:

1. Clone your private BPA3 fork on the HPC cluster.
2. Set up the fresh Python environment described in the official BPA3 `student_guide.md`.

Run job workflow:

1. Submit training through Slurm.
2. Inspect logs and saved models.
3. Copy only the final agent to `models/model.obj`.
4. Test the final agent with `try_agent.py`.
5. Push the final solution to your private fork or download it.

Actual training should run as a **Slurm batch job**, not interactively on the login node.

Connect to the RWTH HPC cluster

The cluster consists of login and compute nodes. Login nodes are used to download code, test small scripts, and start jobs. Compute nodes are for heavy compute tasks. To use them, you schedule a job with [Slurm](#).

WARNING

Do **not** run long training jobs on an HPC login node. Use login nodes only for cloning, editing, installing the environment, checking small commands, and submitting Slurm jobs.

From your local machine, connect via SSH to a **GPU login node**. This is useful for checking CUDA/PyTorch setup before submitting GPU jobs.

```
ssh -l <rwth-id> login23-g-1.hpc.itc.rwth-aachen.de
```

An example RWTH ID would be `ab123456`.

If the connection fails, check whether you are connected to the RWTH network or [RWTH VPN](#).

For a graphical file-transfer login, use an SFTP client such as [FileZilla](#) or [WinSCP](#). Configure it with protocol `SFTP`, host `login23-g-1.hpc.itc.rwth-aachen.de`, port `22`, and your RWTH ID. For larger file transfers, RWTH recommends the dedicated copy login node; see the official [RWTH upload/files transfer documentation](#).

Setup

Clone your BPA3 fork

The HPC has three main file systems; see the official [RWTH file systems overview](#).

- `$HOME` for important source code and configuration. It has backups.
- `$WORK` and `$HPCWORK` for training outputs, logs, and larger files.

To check your file-system quota, use:

```
r_quota
```

Here we use only `$WORK`. Clone your source code from your private repository.

```
cd $WORK
git clone <repo-url> rllbc_bpa3
cd rllbc_bpa3
```

Install the BPA3 environment

Follow the official BPA3 [student_guide.md](#) and the [Pixi installation documentation](#) to install the environment. On the HPC, you can do it via:

```
curl -fsSL https://pixi.sh/install.sh | sh
pixi install --frozen
```

Run a small smoke test

Before submitting a real training job, run a short import and syntax check from the project root. This does **not** start training.

```
pixi run python -m py_compile agent_interface.py util.py training.py try_agent.py
```

```
pixi run python -c "import torch, gymnasium, stable_baselines3; print('CUDA:',  
torch.cuda.is_available())"
```

Submitting jobs

HPC compute nodes

The [RWTH CLAIX cluster](#) has CPU-only compute nodes and GPU compute nodes. Use CPU nodes for small tests, debugging, and non-neural-network workloads. For BPA3 neural-network training, a GPU node is usually preferable because the provided `training.py` automatically uses CUDA when PyTorch can see a GPU.

According to the official [RWTH partitions documentation](#), CLAIX-2023 standard CPU compute nodes use Intel Xeon 8468 Sapphire Rapids CPUs with 96 CPU cores per node. The GPU partition `c23g` consists of nodes with 96 CPU cores and 4 NVIDIA H100 GPUs per node. When you request one GPU with `#SBATCH --gres=gpu:1`, RWTH limits the job to the corresponding quarter of the node: **1 GPU, up to 24 CPU cores, and about 100-122 GB RAM**. One GPU-hour is billed as **24 core-hours**; see the official [RWTH computing time accounting documentation](#).

For the BPA3 GPU example job, request one GPU and the CPU/memory share that belongs to one GPU allocation:

```
#SBATCH --gres=gpu:1  
#SBATCH --cpus-per-task=24  
#SBATCH --mem=100G
```

You can check your current usage and quota with:

```
r_wlm_usage
```

 **How long can you run GPU jobs?** RWTH accounting uses core-hours. A one-GPU job on `c23g` costs about **24 core-hours per wall-clock hour**. RWTH lists a typical student monthly quota of **500 core-hours**; see the official [RWTH HPC introduction slides](#). This corresponds to roughly: $500/24 \approx 20.8$. So, with that quota, a student can run about **20 GPU-hours per month** on one GPU. Your actual quota may differ, so always check `r_wlm_usage`.

 First test your code locally. Only submit the full training run on the cluster once the code starts cleanly, uses the correct environment, and saves to the expected model path.

Set a sensible `#SBATCH --time` limit: long enough that the job is not killed before training finishes, but not so long that a bug or infinite loop consumes your whole quota. A good strategy is to run one short

pilot job on the cluster with low timeout. Then estimate the runtime from the log, and then set the full job time limit with a small safety margin.

Do not request more CPUs, memory, GPUs, or runtime than you need. Larger requests can increase accounting cost and queue waiting time.

Slurm files

A [Slurm](#) file is a shell script that describes **what resources you request** and **what commands should run** once Slurm has allocated a compute node. It has two parts:

1. `#SBATCH` lines at the top: these are read by Slurm before the job starts.
2. normal shell commands below: these are executed on the allocated compute node.

Example structure:

```
#!/usr/bin/env bash

#SBATCH --job-name=bpa3-gpu
#SBATCH --output=logs/%x-%j.out
#SBATCH --error=logs/%x-%j.err
#SBATCH --time=08:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=24
#SBATCH --mem=100G
#SBATCH --gres=gpu:1

set -euo pipefail
cd "${SLURM_SUBMIT_DIR}"
mkdir -p logs runs models
pixi run python training.py
```

Important `#SBATCH` options; see also the official [sbatch documentation](#):

- `--job-name` : readable name shown in `squeue` and used in log file names.
- `--output` : stdout log file. `%x` is replaced by the job name and `%j` by the Slurm job id.
- `--error` : stderr log file. Use this to separate Python errors and warnings from normal output.
- `--time` : maximum wall-clock time. The job is stopped when this limit is reached.
- `--nodes` : number of compute nodes. BPA3 should normally use `1`.
- `--ntasks` : number of tasks/processes. BPA3 should normally use `1`.

- `--cpus-per-task` : CPU cores available to the Python process. With one `c23g` GPU allocation, use up to `24`.
- `--mem` : total memory requested for the job. With one `c23g` GPU allocation, use about `100G`.
- `--gres=gpu:1` : request one GPU. Without this line the job is a CPU job.

The example scripts use `cd "${SLURM_SUBMIT_DIR}"` so that the job runs in the directory from which you called `sbatch`. Therefore submit from the root of your BPA3 repository.

Example Slurm files

We provide two example Slurm files that start training on a CPU or GPU node. They use Pixi by default, i.e. Python commands are run as `pixi run python ...` inside the Slurm job.

```
examples/slurm/submit_train_cpu.sh
examples/slurm/submit_train_gpu.sh
```

Copy them into the root of your BPA3 repository and mark them as executable on the cluster:

```
chmod +x submit_train_*.sh
```

The example scripts create these folders if needed:

```
logs/      # Slurm stdout/stderr files
runs/      # optional per-job run folders
models/    # saved agents/checkpoints
```

The scripts also print useful diagnostic information at the beginning of the log: job id, host name, working directory, Pixi version, Python version, and for GPU jobs the output of `nvidia-smi` plus a PyTorch CUDA check.

Log structure of the Slurm jobs

The example scripts write logs to `logs/` with this pattern:

```
logs/<job-name>-<job-id>.out
logs/<job-name>-<job-id>.err
```

For example:

```
logs/bpa3-gpu-12345678.out
logs/bpa3-gpu-12345678.err
```

Use the `.out` file for normal progress output such as episode rewards, evaluation results, printed

environment information, and WandB messages. Use the `.err` file for Python tracebacks, import errors, CUDA errors, and shell errors.

Useful commands:

```
ls -lh logs  
tail -n 100 logs/bpa3-gpu-*.out  
tail -n 100 logs/bpa3-gpu-*.err
```

If a job disappears from `squeue --me`, it has either finished or failed. Then inspect the latest `.out` and `.err` files.

Other ways to submit a job

The normal and recommended way is:

```
sbatch submit_train_gpu.sh
```

You can override some `#SBATCH` options directly on the command line without editing the file:

```
sbatch --time=01:00:00 submit_train_gpu.sh  
sbatch --job-name=bpa3-seed1 submit_train_gpu.sh
```

You can also override the training command used by the example scripts:

```
export BPA3_TRAIN_CMD="pixi run python training.py"  
sbatch submit_train_gpu.sh
```

This is useful if you later add command-line arguments to `training.py`:

```
export BPA3_TRAIN_CMD="pixi run python training.py --seed 2 --total-timesteps 1000000"  
sbatch submit_train_gpu.sh
```

Only do this after you have implemented those arguments in `training.py`. The provided `training.py` does not parse them by default.

Submit a training job

From the root of your BPA3 repository:

```
sbatch submit_train_gpu.sh
```

Check the queue:

```
squeue --me
```

Inspect logs:

```
ls -lh logs
tail -n 100 logs/bpa3-gpu-*.out
tail -n 100 logs/bpa3-gpu-*.err
```

The Slurm logs contain the console output of `training.py`. The provided BPA3 training script prints episode rewards and evaluation results there. If `track = True` in `training.py`, [Weights & Biases](#) logging is also enabled. In that case, make sure you have logged in to WandB with [wandb login](#) on the cluster before submitting the job, or set `track = False` in `training.py` if you do not want online logging. WandB run metadata is usually placed in a local `wandb/` folder in the repository, while the online metrics appear in the WandB project configured in `training.py`, currently `RLLBC_BPA3`.

Training results are placed in the repository folders used by the script:

```
logs/      # Slurm stdout/stderr logs
runs/      # optional run directory created by the Slurm script
models/    # saved agents/checkpoints
wandb/     # local WandB metadata if WandB tracking is enabled
```

The provided `training.py` saves the best and last model under names such as:

```
models/ddpg_benchmark_final_best.obj
models/ddpg_benchmark_final_last.obj
```

Then check that the model expected by the evaluation exists:

```
ls -lh models/model.obj
pixi run python try_agent.py
```

If your training script writes intermediate models somewhere else, copy only the selected final model to `models/model.obj` before submission.

Common mistakes

Running training on the login node

Do not do this. Use Slurm.

Forgetting environment activation in the Slurm script

The Slurm job starts in a fresh shell. Activate the environment inside the script, or use `pixi run` directly in the training command.

Passing unsupported arguments to `training.py`

The provided `training.py` should not be assumed to support options such as `--seed`, `--total-steps`, `--device`, or `--output-dir`. Add such options yourself before using them in Slurm scripts.

Saving the model to the wrong path

The final model must be saved as:

```
models/model.obj
```

Importing extra packages in `agent_interface.py`

The evaluation environment may not contain packages that you installed only for training. Keep `agent_interface.py` compatible with the provided environment.

Committing large files

Do not commit logs, videos, replay buffers, or intermediate checkpoints.

Trusting a single training run

RL performance can vary between seeds. Compare multiple runs before drawing conclusions.

Official documentation and websites

Consult the official documentation for current details:

- [RWTH HPC Getting Started](#)
- [RWTH Submitting Jobs](#)
- [RWTH Job Parameters](#)
- [RWTH Computing Time Accounting](#)
- [RWTH File Systems Overview](#)
- [RWTH JupyterLab and Slurm](#)
- [RWTH HPC introduction slides](#)
- [Slurm `sbatch` documentation](#)
- [Pixi installation documentation](#)

- [Weights & Biases documentation](#)